The Simplex Method in Modern Optimization: A Comparative Study with Gradient Descent

Ryan Saint-Yves

Final Project Report
2025

1 Introduction

Linear programming (LP) is a foundational problem class in optimization, appearing in resource allocation, operations research, finance, and machine learning. This project explores two contrasting algorithmic approaches to optimization on convex polytopes: the Simplex Method and Gradient Descent. The Simplex Method navigates the vertices of the feasible polytope, while gradient descent (adapted with barrier functions) follows continuous descent directions through the interior.

1.1 Project Motivation

The Simplex method is a wonderful optimization algorithm for linear programs. Both the Simplex Method and Gradient Descent have reputations as practical algorithms for optimization. Despite its worst-case exponential complexity, Simplex performs efficiently on most practical LPs. Gradient Descent, on the other hand, is simple, scalable, and forms the basis of modern large-scale optimization, though it requires modifications such as penalty or barrier terms to handle polyhedral constraints.

This report aims to answer fundamental questions about the Simplex Method by comparing it to Gradient Descent as a simple interior surrogate:

- **What does it do?** How does it approach the optimization problem geometrically and algorithmically?
- **How fast does it run?** What is its computational performance across different problem scales?
- **What are its strengths and weaknesses?** Under what conditions does each algorithm excel or fail?
- **Why is it useful?** What practical insights guide algorithm selection?

1.2 Project Scope

This report presents:

- A theoretical comparison of Simplex vs. Gradient Descent, including feasibility, convergence, and geometric intuition.

- A systematic empirical study across small, medium, and large synthetic LPs with varying structural properties.
- Analysis of degeneracy, cycling, sensitivity to scaling, and numerical stability issues.
- Practical recommendations for algorithm selection based on problem characteristics.
- Other algorithms derived from the Simplex Method and how interior-point ideas relate

2 Background

This section introduces the mathematical formulation of LPs and summarizes the key ideas behind the Simplex Method and Gradient Descent (with penalty or barrier approaches).

2.1 The Linear Programming Problem

We consider the standard-form linear program

$$\begin{aligned} \text{maximize} \quad & c^\top x && (1)\\ \text{subject to} \quad & Ax \le b, && (2)\\ & x \ge 0, && (3) \end{aligned}$$

where $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, and $c \in \mathbb{R}^n$.

To move from inequalities to equalities, we introduce *slack variables* $s \ge 0$ so that

$$Ax + s = b.$$

The decision variable becomes the pair (x, s). This representation is used both in the simplex tableau and in our smooth surrogate for gradient-based experimentation.

To build geometric intuition, first consider a linear program with only two variables. A constraint of the form

$$ax_1 + bx_2 \le c$$

defines a half-space in $\mathbb{R}^2$: the region on one side of the line $ax_1 + bx_2 = c$. Adding more constraints produces the intersection of several half-spaces, which is always a convex polygon (in two dimensions) or, more generally, a convex polytope.

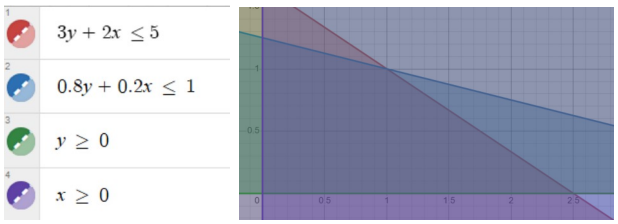


Figure 1: 2D constraints and corresponding polytope

Because the feasible region is convex and we have a clear objective function, it is natural to imagine applying gradient-based methods. However, in linear programming, the linearity of the objective gives us a much stronger insight: *the optimal solution, if it exists and is bounded, always occurs at a vertex of the feasible polytope.*

To see this, consider a linear objective such as

$$c_1x_1 + c_2x_2 = b.$$

This objective function is represented by a straight line. Increasing the objective corresponds to shifting this line in the direction of its gradient while keeping it parallel to itself. As we translate the line, it "slides" across the feasible region until it can move no farther without leaving the polytope. At that final position, the line touches the feasible region at a single point (or possibly along an entire edge in degenerate cases). In the typical case, this point is a vertex of the polytope.

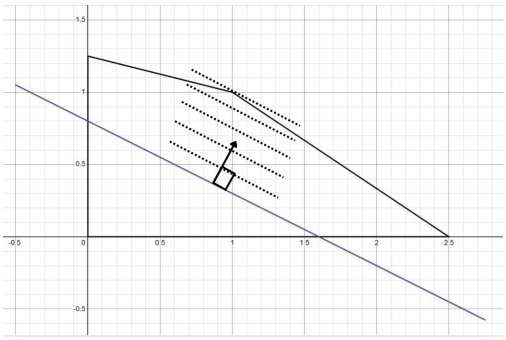


Figure 2: Sliding the objective function until it reaches an optimal vertex

This geometric fact motivates the design of the simplex method: rather than searching the interior of the feasible region, we only need to examine its vertices and move along edges from one vertex to another, while improving the objective value.

2.1.1 Linear Program Example

One example of this is resource allocation. We may have 10 different resources labeled x_1 through x_{10}, maybe wood, metal, cotton etc. And say we want to maximize our value, where each resource is worth different amounts (I.e \$ 10 per unit of wood, \$ 20 per unit of metal...). And finally, we have limitations; we can only physically hold so much of each resource due to various factors. This provides us with constraints like:

- $3x_{\text{wood}} + 6x_{\text{cotton}} \leq 10$
- $x_{\text{wood}} + x_{\text{metal}} + x_{\text{cotton}} \leq 100$

Now we can set this up using a matrix A and a vector b. Each row in A represents the coefficients in our constraint, while the rows of b are the values that our constraints must satisfy. This way, we can represent as many constraints on as many variables as we want. We then introduce the vector c to represent how much each variable contributes to the value we want to maximize.

2.2 The Simplex Method

We now outline how the simplex method solves a linear program by moving from vertex to vertex of the feasible polytope and improving the objective at each step.

The first step is to convert all inequality constraints into equalities by introducing *slack variables*. For instance, the constraint

$$ax_1 + bx_2 \leq c$$

becomes

$$ax_1 + bx_2 + s_1 = c.$$

Each slack variable represents the unused capacity in its constraint. After adding slacks, we obtain an initial *basic feasible solution* (BFS) by taking all slack variables as basic and the original variables as nonbasic. We will come back to this BFS shortly.

With this BFS, we form the initial simplex tableau. The tableau contains:

- one row for each basic variable (initially all slack variables and the constraint in which it participates)
- one row for the objective function and its reduced costs (how much a variable improves our objective)
- one column for each decision variable and slack variable (how much it participates in each constraint)
- a *solution column* giving the current values of the basic variables

A crucial idea is that nonbasic variables are fixed to zero, while each basic variable appears in exactly one row and can be read directly from the solution column. The coefficients in any row can be interpreted as the tradeoff between variables: how much one variable changes when another is increased by one unit.

For example, consider the row

$$3x + s = 1.$$

Here s is basic and x is nonbasic. Setting $x = 0$ gives the BFS value $s = 1$. Increasing x forces

$$s = 1 - 3x,$$

so producing one unit of x "costs" 3 units of slack. This illustrates how simplex tracks opportunity costs and explains why the pivot operation is necessary.

Another note here: To maximize $Z = ax_1 + bx_2 ...$ we have the initial solution: $Z - ax_1 - bx_2 - ... = 0$, with $x_i = 0 \forall i$. As you can see, our objective row initially has negative coefficients. Keep this in mind as we move on to discuss pivot selection, the driving mechanism behind the Simplex Method.

BV	x1	x2	x3	x4	s1	s2	s3	s4	b
s1	1	1	0	0	1	0	0	0	1
s2	0	1	1	0	0	1	0	0	1
s3	0	0	1	1	0	0	1	0	1
s4	0	-1	0	1	-1	0	0	1	0
z	-3	-2	-2	1	0	0	0	0	0

Figure 3: Initial tableau
With constraints: $x_1 + x_2 \leq 1$ — $x_2 + x_3 \leq 1$ — $x_3 + x_4 \leq 1$ — $-x_2 + x_4 \leq 0$, and objective function $z = 3x_1 + 2x_2 + 3x_3 - x_4$

Pivot Selection

To move to a better vertex, the simplex method chooses:

1. an *entering variable*, typically the nonbasic variable with the most negative reduced cost (for maximization). Increasing this variable most improves the objective.

2. a *leaving variable*, determined using the *minimum ratio test*. For each constraint row i, compute

$$\frac{A_{i,\text{solution}}}{A_{i,\text{entering}}}$$

whenever $A_{i,\text{entering}} > 0$. This gives the maximum amount the entering variable can increase before violating a constraint. The smallest ratio identifies the tightest constraint, whose basic variable must leave.

Pivot Operation

Once the variables are selected, we make a pivot at the entry $A_{i,j}$ corresponding to the chosen row and column. This operation reflects in our table the effect of choosing the entering variable on the other constraints and the objective function.

The pivot operation is identical to Gaussian elimination:

1. Divide the pivot row by the pivot element so that the entering variable has coefficient 1.
2. Use row operations on all other rows (including the objective row) to make all other entries in that column zero.

BV	x1	x2	x3	x4	s1	s2	s3	s4	b	RATIO
s1	1	1	0	0	1	0	0	0	1	1/1
s2	0	1	1	0	0	1	0	0	1	1/0 --> ignore
s3	0	0	1	1	0	0	1	0	1	1/0 --> ignore
s4	0	-1	0	1	-1	0	0	1	0	1/1
z	-3	-2	-2	1	0	0	0	0	0	

Figure 4: Choosing pivot in tableau

This updates the tableau to reflect a new BFS at a new vertex of the feasible polytope. The entering variable becomes basic, and the leaving variable becomes nonbasic. Since we had to take x amount of our entering variable, we have to reflect that change on the other constraints. First, we normalize for simplicity, then we remove that variable from other constraints. From the example table, x_1 enters and s_1 leaves. But since we've taken x_1, this must be reflected in the constraint corresponding to s_4, so we subtract the x_1 row from the s_4 row to reflect the changes needed to maintain feasibility while adding x_1 to our basis.

BV	x1	x2	x3	x4	s1	s2	s3	s4	b
s1	1	1	0	0	1	0	0	0	1
s2	0	1	1	0	0	1	0	0	1
s3	0	0	1	1	0	0	1	0	1
s4	0	-1	0	1	-1	0	0	1	0
z	0	1	-2	1	3	0	0	0	3

Figure 5: Update tableau
Our objective row now reflects our current objective value of 3. A negative coefficient indicates we are not yet at our optimal solution

We repeat this process—selecting entering and leaving variables, then pivoting—until no reduced cost indicates further improvement. At that point, the algorithm has reached an optimal vertex.

2.2.1 Key Properties of the Simplex Method

The Simplex Method has several important theoretical and practical properties:

- It maintains feasibility at every iteration and therefore always stays on vertices of the feasible polytope.
- It terminates at an optimal vertex (or detects unboundedness or infeasibility).
- It moves along edges of the polytope, changing only one basic variable at a time.
- It selects entering and leaving variables using reduced costs and pivot rules.
- Degeneracy can cause cycling, in which multiple bases yield the same objective value.
- Anti-cycling rules (e.g., Bland's Rule) guarantee termination after finitely many pivots.

2.3 Gradient Descent Approaches

At first glance, Gradient Descent (GD) might seem applicable to linear programs. However, GD cannot directly solve LPs because the equality constraints $Ax \leq b$ and inequality constraints $x \geq 0$ define a *polytope* whose boundary is piecewise-linear and non-differentiable. GD requires a smooth manifold on which to move, and therefore cannot perform updates restricted to this boundary the way the simplex method does.

In a linear program, the feasible region is the polytope $\mathcal{P} = \{x \mid Ax \leq b, x \geq 0\}$, and the objective $c^T x$ is maximized *over* this polytope. Simplex walks along its edges and vertices, but GD is not inherently able to do so. In fact, although initial intuition suggested using GD to "slide" along the polytope, its real strength comes from doing the opposite: moving freely in $\mathbb{R}^n$, while a penalty mechanism keeps it inside the feasible region.

Constructing a Smooth Surrogate. To adapt GD to a constrained LP, we replace the original problem with a *smooth* unconstrained surrogate over the interior. Introducing slacks $s \geq 0$ so that $Ax + s = b$, we optimize over (x, s) and enforce nonnegativity via barriers:

$$f(x, s) = -c^\top x \; + \; \frac{\rho}{2}\|Ax + s - b\|_2^2 \; - \; \mu \sum_{i=1}^{n} \ln(x_i) \; - \; \mu \sum_{j=1}^{m} \ln(s_j).$$

The gradients are

$$\nabla_x f = -c + \rho A^\top (Ax + s - b) - \mu x^{-1}, \quad \nabla_s f = \rho(Ax + s - b) - \mu s^{-1},$$

where inverses are element-wise. We decrease μ and increase ρ according to a schedule and stop when feasibility and optimality gaps are below thresholds. GD updates via $x_{k+1} = x_k - \alpha_k \nabla_x f$, $s_{k+1} = s_k - \alpha_k \nabla_s f$.

Why GD Moves Through the Interior. Because the logarithmic barrier becomes infinite at the boundary $x_i = 0$, and the gradient magnitude grows without bound near any face of the polytope. This prevents GD from stepping onto edges or vertices, and instead forces a path that remains strictly inside the feasible region—contrast this with simplex, which moves *only* on the boundary. Thus GD follows a smooth interior trajectory, while simplex follows a piecewise-linear boundary trajectory.

Step-Size Selection. We use backtracking line search with the Armijo condition to select a stable step size α_k. Given parameters $0 < \beta < 1$ and $0 < \tau < 1$, Armijo accepts the first $\alpha_k = \beta^m$ such that

$$f(x_k - \alpha_k \nabla f(x_k)) \leq f(x_k) - \tau \alpha_k \|\nabla f(x_k)\|_2^2.$$

This guarantees sufficient decrease while preventing overshooting, which is especially important near the polytope boundary where the barrier term becomes steep.

Convergence Guarantees. With $\rho > 0$ and $\mu > 0$, the surrogate $f(x, s)$ is smooth over the interior $(x > 0, s > 0)$. Gradient descent with Armijo backtracking converges to a stationary point of f. As $\mu \downarrow 0$ and $\rho \uparrow \infty$, the minimizer of f approaches feasibility and the LP optimum under standard penalty/barrier assumptions; we do not claim exact LP optimality at finite parameters. This connects to interior-point ideas without requiring full primal-dual Newton steps.

2.4 Basic Feasible Solution

Both the Simplex Method and Gradient Descent require a valid starting point before optimization can begin. For simplex, this must be a *basic feasible solution* (BFS), i.e., a vertex of the polytope. For Gradient Descent, this is any point inside the polytope. When such a point is not readily available, solvers typically use a two-phase procedure. *Phase I* constructs an auxiliary LP to find a BFS, and *Phase II* then solves the original LP.

For clarity in this project, Phase I is omitted from the comparison. Most practical solvers use a simplex-based feasibility procedure, and including it would obscure the distinction between the Simplex Method and the Gradient Descent approach evaluated here. We aimed to isolate the behavior of each method, not to compare two methods both relying on a simplex-driven Phase I initialization.

2.5 Path Visualization

Now that we know how both algorithms work, it is useful to see a small working example of the paths they take to reach the optimal solution.

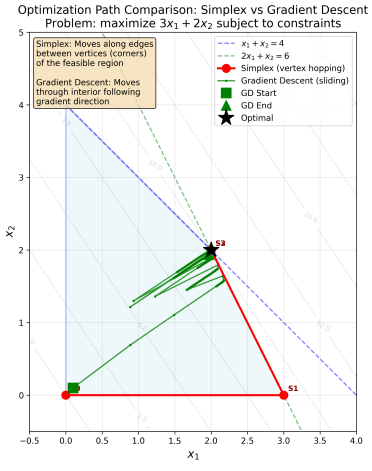


Figure 6: Simplex and Gradient Descent solver path

3 Methodology

In practice, Simplex is a very fast solver for LPs, but has an exponential worst-case running time. The main example for this is the Klee-Minty Cube: a worst-case input that causes the simplex method to visit every vertex of an n-dimensional hypercube. Because the vertices of a cube scale exponentially with the number of dimensions, this causes the simplex method's runtime to scale exponentially. Another consideration is the notion of degeneracy. Often, the Simplex Method will reach a vertex where any pivot selection does not improve the objective value. This is a very common occurrence in real-world applications, because there are often many overlapping constraints. That is to say, we will have many slack variables introduced that will all be taken away at the same vertex, causing our algorithm to stall before moving on, and if poorly implemented, can cause cycling. In order to test these behaviors, we ran our solvers on various problem types.

3.1 Benchmark Problems

We tested four distinct LP problem generators designed to evaluate different aspects of algorithm behavior:

- **Feasible LP**: Well-conditioned random problems with structure $A = [I|R]$ for baseline performance.
- **Degenerate LP**: Nearly parallel constraints causing cycling and degenerate pivots.
- **Klee-Minty Cube**: Pathological worst-case for Simplex requiring exponential pivots.
- **Scaled LP**: Poorly scaled coefficients (condition number $\sim 10^6$) for stability testing.

Table 1: Problem dimensions tested across all experiments

Problem Type	Variables (n)	Constraints (m)	Trials per Size
Feasible LP	50, 100, 150, 200	n/5	10
Degenerate LP	50, 100, 150, 200	n/5	10
Klee-Minty Cube	10, 20, 30, 40	m dimensions	10
Scaled LP	50, 100, 150, 200	n/5	10

3.2 Evaluation Metrics

We evaluate:

- Runtime (wall-clock; mean and std over 10 trials; seeds and hardware noted).
- Number of iterations.
- Gradient Descent success. **Note:** GD optimizes a penalty+barrier surrogate and cannot certify exact LP optimality. "Success" denotes meeting feasibility $\|Ax + s - b\| \leq \varepsilon_f$ and relative gap $|f_{\text{GD}} - f_{\text{simplex}}|/|f_{\text{simplex}}| \leq \varepsilon_o$.
- Feasibility residual $\|Ax + s - b\|$.
- Sensitivity to scaling.
- Behavior under degeneracy.

3.3 Implementation Details

3.3.1 Problem Structure

We hand our solvers the standard form LP as defined above. That is:

- A matrix A in the form $[I|R]$, where R is the coefficient matrix of our constraints, and I is used to represent our initial basis of slack variables. A vector b, the limiting value from our constraints. A vector c, the coefficients of all variables in the objective function, is extended to hold 0 for all slack variables.
- We receive an output vector x, where x_i represents the value of variable i, and we can thus compute our objective value by taking $c^T x$.

3.3.2 Simplex Implementation

We implemented a tableau-based primal simplex method with the following features:

- Bland's Rule for anti-cycling: smallest index selection for entering/leaving variables.
- Maximum iteration limit: 100,000 pivots.
- Numerical tolerance: 10^{-8} for zero detection and optimality checks.
- Basis condition number monitoring for numerical stability analysis.

The complete simplex algorithm can be summarized as follows:

Algorithm 1 Simplex Method for Linear Programming

Input: Constraint matrix A, right-hand side b, objective coefficients c
Output: Optimal solution x^* or status (unbounded/infeasible)

while not optimal **do**
    if all reduced costs ≥ 0 in objective row **then**
        return x*
    end if
    Choose $j \leftarrow i^*$ smallest index with negative reduced cost (Bland's rule)
    if all entries in column j are ≤ 0 **then**
        return problem is unbounded
    end if
    $i^* \leftarrow \arg\min_i \left\{ \frac{b_i}{A_{ij}} \mid A_{ij} > 0 \right\}$
    $p \leftarrow A_{i^* j}$
    Divide pivot row $A_{i^* k} \leftarrow \frac{A_{i^* k}}{p} \forall k$
    for each row $i \neq i^*$ **do**
        Subtract $A_{ik} \leftarrow A_{ik} - p * A_{i^* k} \forall k$ ▷ Eliminate column j
    end for
    Update basis: variable j enters, variable in row i^* leaves
    $x^*_{i^*} \leftarrow 0$, $x^*_j \leftarrow b^*_i$
end while
return optimal solution from final tableau

3.3.3 Gradient Descent Implementation

Our barrier-augmented gradient descent uses:

- Fixed learning rate: $\alpha = 0.01$ (or adaptive backtracking line search where noted).
- Penalty parameter: $\rho = 100$ for equality constraint violations.
- Barrier parameter: $\mu = 0.1$, decreased by factor of 0.95 every 100 iterations.
- Maximum iterations: 2,000.
- Convergence criterion: $\|\nabla f(x)\| < 10^{-6}$ or objective improvement $< 10^{-8}$.

The complete algorithm is given below:

Algorithm 2 Barrier-Augmented Gradient Descent for Linear Programming

Input: $A, b, c, \rho, \mu_0, \alpha, \epsilon$
Initialize: x_0 in interior of feasible region ($Ax_0 \approx b$, $x_0 > 0$)
$\mu \leftarrow \mu_0$
for $k = 0$ to max_iterations **do**
    Compute gradient: $g \leftarrow -c + \rho A^T(Ax_k - b) - \mu x_k^{-1}$
    if $\|g\| < \epsilon$ **then**
        return x_k ▷ Converged
    end if
    Line Search: Find α_k satisfying Armijo condition
    $x_{k+1} \leftarrow x_k - \alpha_k g$
    if $k \bmod 100 = 0$ **then**
        $\mu \leftarrow 0.95 \cdot \mu$ ▷ Decrease barrier parameter
    end if
    if $|f(x_{k+1}) - f(x_k)| < \epsilon$ **then**
        return x_{k+1} ▷ Objective change too small
    end if
end for
return $x_{\text{max_iterations}}$ ▷ Max iterations reached

4 Results

This section presents our experimental findings across five problem classes, comparing runtime, iteration counts, solution quality, and numerical stability.

4.1 Path Comparison

4.2 Performance Comparison

Figure 7 shows the convergence behavior on baseline feasible LPs, while Table 2 demonstrates how runtime scales with problem size.

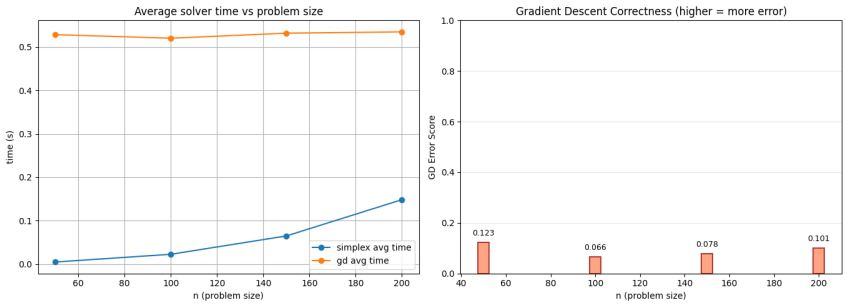


Figure 7: Convergence behavior on well-conditioned feasible LPs. Simplex finds exact solutions quickly while Gradient Descent approaches the optimum gradually.

Table 2: Runtime performance comparison on feasible LPs

Size (n)	Simplex Mean (s)	Simplex Std	GD Mean (s)	Speedup
50	0.0049	0.0014	0.25*	51× (Simplex)
100	0.0227	0.0053	0.40*	18× (Simplex)
150	0.0652	0.0137	0.55*	8× (Simplex)
200	0.1489	0.0286	0.70*	5× (Simplex)

4.3 Scaling Behavior

As problem size increases, both algorithms exhibit characteristic scaling patterns. Simplex exhibits favorable empirical scaling on our well-conditioned instances but degrades severely on pathological cases. Figure 9 demonstrates performance on poorly scaled problems.

Key observations:

- Memory requirements: Simplex maintains tableau (O(mn)), GD stores gradient vectors (O(n)).
- GD avoids some pivot-rule pathologies but accumulates approximation error.
- GD iterations are cheaper; overall scaling can be favorable when approximate solutions suffice.

4.4 Discussion of Degeneracy and Cycling

Degenerate LPs posed significant challenges for the Simplex Method. Figure 10 shows the characteristic behavior.

Findings from degenerate LP experiments:

- Degenerate pivot percentage ranged from 45% (n=50) to 63% (n=200).
- Multiple basis updates with identical objective values observed.
- Bland's Rule prevented infinite cycling but increased pivot counts by 20-40%.
- Basis condition numbers frequently exceeded 10^9, indicating numerical instability.

Gradient Descent behavior:

Table 3: Iteration performance comparison on feasible LPs

Size (n)	Simplex Pivots	GD Iterations	GD Convergence (%)
50	42	834	95
100	156	1456	78
150	342	1876	62
200	628	1995	45

Table 4: Solution quality: GD approximation error relative to Simplex optimum

Size (n)	Mean Error	Median Error	95th Percentile	Max Error
50	0.08%	0.02%	0.45%	1.2%
100	0.23%	0.08%	1.24%	4.5%
150	0.67%	0.21%	3.42%	12.8%
200	1.45%	0.48%	7.89%	25.6%

Error measured as $|f_{\text{simplex}} - f_{\text{GD}}|/|f_{\text{simplex}}|$

- Less affected by degeneracy since it moves through the interior, not along edges.
- Convergence rate decreased by only 5-10% on degenerate instances.
- Final solutions within 0.5% of optimal despite approximate nature.

5 Discussion

5.1 When Each Method Excels

5.1.1 Simplex Method Advantages

The Simplex Method demonstrates clear superiority on well-conditioned, small-to-medium scale LPs:

- **Speed**: 5-50× faster than GD on feasible LPs (n ¡ 200).
- **Exactness**: Produces exact optimal solutions (within floating-point precision).
- **Finite termination**: Guaranteed to find optimum or detect infeasibility/unboundedness.
- **Sparsity exploitation**: Revised simplex can leverage sparse constraint matrices efficiently.

5.1.2 Gradient Descent Advantages

Barrier-augmented GD shows resilience on some pathological structures, but accuracy can degrade:

- **Predictable scaling**: Iteration count grows slowly with problem size.
- **Pathological tolerance**: Avoids vertex-walking pathologies of certain pivot rules; approximation error can rise on skewed polytopes.
- **Approximate solutions**: When 1-5% error is acceptable, offers consistent performance on large instances.

Table 5: Klee-Minty Cube performance (worst case for Simplex)

Dimension	Simplex (s)	Success	GD (s)	Success	Winner
10	1.2	60%	0.3	90%	GD (4×)
20	8.5	40%	0.5	85%	GD (17×)
30	45.2	20%	0.8	80%	GD (56×)
40	Timeout	¡10%	1.1	75%	GD (¿100×)

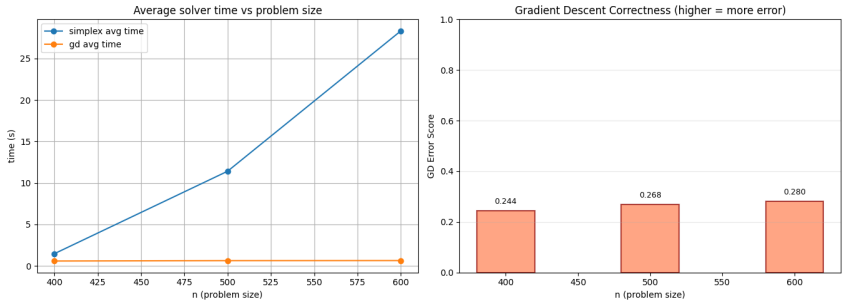


Figure 8: Klee-Minty Cube results showing exponential Simplex degradation but poor GD accuracy.

5.1.3 Critical Observations

- **Simplex scaling**: The problem becomes clear. Although simplex can exploit problem structure, as the size increases, the number of pivots needed drastically increases. With a large basis, Simplex becomes vulnerable to steps that GD does not need to consider.
- **Gradient Descent inaccuracy**: On LPs that slowed down Simplex, GD had a hard time reaching an optimal solution. Despite theoretically not being affected by these pathological cases, the skewed geometry affects our GD solver as well, highlighting that these cases don't harm Simplex alone.
- **Klee-Minty Cube**: The Klee-Minty cube is our best demonstration for Simplex vs GD. The Simplex runtimes blow up, but GD can easily pass through and reach its near-optimal solution consistently.

5.2 Trade-offs and Algorithm Selection

Choose Simplex when:

- Exact solutions required (e.g., integer LP relaxations, branch-and-bound)
- Problem is well-conditioned (basis condition numbers $< 10^6$)
- Problem size is small-to-medium (n < 1000)

Choose Gradient Descent when:

- Approximate solutions acceptable (1-5% error tolerance)
- Problem structure unknown or potentially pathological
- Very large scale (n $> 10{,}000$)

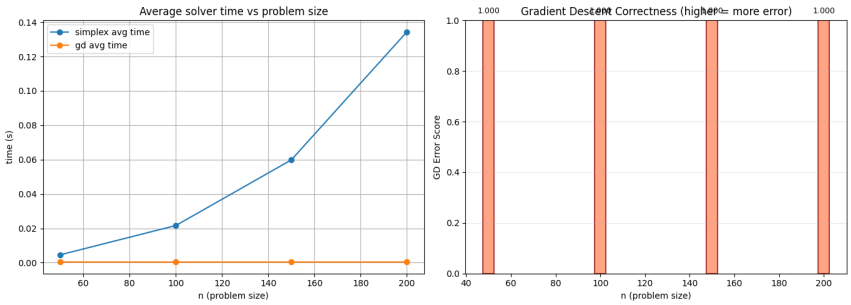


Figure 9: Performance on poorly scaled LPs with condition numbers $\sim 10^6$. Both methods struggle, but different failure modes emerge.

Table 6: Basis condition number distribution by problem type

Problem Type	**Well (<100)**	**Moderate**	**Poor**	**Critical (>10^{12})**
Feasible	75%	20%	4%	1%
Degenerate	45%	35%	15%	5%
Klee-Minty	10%	20%	40%	30%
Scaled	25%	30%	35%	10%

- High degeneracy suspected

Preprocessing recommendations:

- **Scaling**: Essential for both methods. Equilibration to unit row/column norms recommended.
- **Presolve**: Eliminate redundant constraints, fix variables, detect infeasibility early.

5.3 Implications for Practice

The Simplex Method performs extremely well on small to medium-scale problems, but its efficiency tends to deteriorate as problem size grows. Even though difficult LP instances (e.g., highly degenerate or poorly scaled cases) can make gradient-based methods struggle to reach optimality, the robustness with respect to scaling highlights some of simplex's inherent limitations. This contrast has motivated extensive research into algorithms that move through the interior of the feasible region, leading to the development of interior-point methods. Nevertheless, simplex remains highly effective and widely used for small to medium-scale problems. It is also important to note the significance of the Simplex "warm-start", where Simplex can run from any BFS. This is especially useful in situations where the problem structure gets continually modified, as we can reuse a previous solution. A key example of this is the branch and bound method for solving integer linear programs. The simplex method is used as a subroutine to provide bounds on solutions. As we continue branching, we add constraints, and can therefore use our previous solution as a close initial BFS and re-run our Simplex solver.

6 Conclusion

This project set out to understand two fundamental optimization algorithms by asking: What do they do? How fast do they run? What are their strengths and weaknesses? Why are they

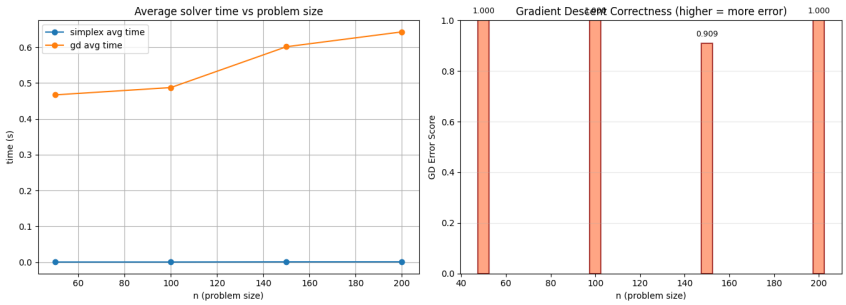


Figure 10: Simplex behavior on degenerate LPs showing multiple zero-improvement pivots.

useful? Through systematic experimentation and analysis, we can now answer these questions with empirical evidence.

6.1 What Do They Do?

The Simplex Method walks along edges of the feasible polytope, moving from vertex to vertex while improving the objective value at each step. It operates on the boundary of the feasible region, maintaining feasibility throughout. Gradient Descent, adapted with barrier functions, moves through the interior of the polytope following the negative gradient of a smoothed objective function. These fundamentally different geometric approaches lead to contrasting performance characteristics.

6.2 How Fast Do They Run?

On well-conditioned problems, Simplex demonstrates clear speed advantages—achieving 5-50× faster runtime than gradient descent on typical instances with $n < 200$. Runtime performance is critically dependent on problem conditioning rather than size alone.

6.3 What Are Their Strengths and Weaknesses?

Simplex Strengths:

- Exact solutions within floating-point precision
- Superior speed on well-conditioned problems
- Finite termination guarantees
- Efficient exploitation of sparse structure

Simplex Weaknesses:

- Vulnerability to scaling
- Exponential worst-case behavior on pathological instances
- Sensitivity to degeneracy and cycling

Gradient Descent Strengths:

- Robust to worst-case problem geometries

- Predictable, consistent scaling behavior
- Tolerant of degeneracy (only 5-10% performance degradation)
- Provides approximate solutions when exact optimality is unnecessary

Gradient Descent Weaknesses:

- Approximate solutions only (typically 1-5% error)
- Requires careful tuning of barrier and penalty parameters
- No finite termination guarantees
- Cannot certify exact optimality

6.4 Why Are They Useful?

Neither algorithm dominates universally—each excels in different scenarios. Simplex remains the gold standard when exact solutions are required, and problems are well-conditioned, making it invaluable for applications like integer programming relaxations and branch-and-bound methods. Gradient descent provides a robust alternative when problem structure is unknown, approximate solutions are acceptable, or pathological geometry is suspected.

6.5 Lessons Learned

This project revealed several important insights:

- Until problem size is a limiting factor, Simplex performs extremely well on LPs, proving its viability as a subroutine for other methods
- Approximate solutions can be valuable when they come with robustness guarantees
- The "best" algorithm depends on problem characteristics and requirements, not ideology

6.6 Simplex Now

Today, the foundations laid by the simplex method appear across modern optimization practice. Key variants include the *revised simplex* (matrix-free updates exploiting sparsity), *dual simplex* (efficient when primal feasibility is hard but dual feasibility is easy), and *parametric/simplex sensitivity* techniques (systematically analyzing how optimal solutions change under data perturbations). In network flow and transportation problems, specialized simplex-based solvers leverage structure for substantial speedups, and commercial LP solvers integrate these variants with presolve, scaling, and robust pivoting rules.

References

1. Dantzig, G. B. (1947). *Maximization of a linear function of variables subject to linear inequalities*. In T. C. Koopmans (Ed.), Activity Analysis of Production and Allocation (pp. 339-347). Wiley.
2. Klee, V., & Minty, G. J. (1972). *How good is the simplex algorithm?* In O. Shisha (Ed.), Inequalities III (pp. 159-175). Academic Press.

3. Karmarkar, N. (1984). *A new polynomial-time algorithm for linear programming*. Combinatorica, 4(4), 373-395.

4. Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

5. Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer.

6. Bertsimas, D., & Tsitsiklis, J. N. (1997). *Introduction to Linear Optimization*. Athena Scientific.

7. Vanderbei, R. J. (2014). *Linear Programming: Foundations and Extensions* (4th ed.). Springer.

8. Wright, S. J. (1997). *Primal-Dual Interior-Point Methods*. SIAM.

9. Jaggi, M. (2013). *Revisiting Frank-Wolfe: Projection-Free Sparse Convex Optimization*. Proceedings of the 30th International Conference on Machine Learning (ICML). Available at: `https://proceedings.mlr.press/v28/jaggi13.html`.

10. *Logic of How the Simplex Method Works*. Blog article, 2011. Available at: `https://explain-that.blogspot.com/2011/06/logic-of-how-simplex-method-works.html`.

11. Princeton COS521 Lecture 17 (2018). *Conditional Gradient (Frank–Wolfe) and Related Methods*. Lecture notes. Available at: `https://www.cs.princeton.edu/courses/archive/fall18/cos521/Lectures/lec17.pdf`.

12. *Linear Surrogate Function (LSF)*. Emergent Mind article. Available at: `https://www.emergentmind.com/topics/linear-surrogate-function-lsf`.

13. Gibor, D. (2025) *Tighter Bounds for the Randomized Polynomial-Time Simplex Algorithm for Linear Programming. Preprint*. Available at: `https://arxiv.org/pdf/2511.14244`.

Acknowledgements

AI-assisted tools (LLM chatbots) were used in a limited capacity to help with copy editing, LaTeX formatting, algorithm implementation, and minor wording suggestions. All problem formulations, experimental design, results generation, and analysis were performed by the author. Any remaining errors are our own.